Fuzzy Aggregators in Practice: Meta-Model and Implementation

Paolo Fosci and Giuseppe Psaila(✉)

University of Bergamo, Viale Marconi 5, 24044 Dalmine, BG, Italy
{paolo.fosci,giuseppe.psaila}@unibg.it
http://www.unibg.it

Abstract. The *J-CO* Framework is a tool designed to provide analysts and data engineers with the capability to perform complex transformations on *JSON* databases through its query language, named *J-CO-QL*$^+$.

Fuzzy aggregators are a powerful tool for decision making, when there is the need to combine together the evaluations of a group of experts to form a generalized opinion. Since soft querying can support decision making, it is straightforward to consider the introduction of the concept of fuzzy aggregation into *J-CO-QL*$^+$.

This paper proposes a generic meta-model to define fuzzy aggregators, and novel *J-CO-QL*$^+$ constructs to define user-defined fuzzy aggregators and use them in practice. A plausible case study shows that the meta-model can be effectively implemented within a stand-alone query language (*J-CO-QL*$^+$) that is supported by a tool (the *J-CO* Framework) that is available for the community.

Keywords: Generic model for fuzzy aggregators · The *J-CO* Framework · User-defined fuzzy aggregators in *J-CO-QL*$^+$

1 Introduction

The *J-CO* Framework is a software tool under development at University of Bergamo (Italy), whose goal is to provide analysts and data engineers with the capability to perform complex transformations and querying of *JSON* data sets, possibly stored within *JSON* document stores (i.e., *NoSQL* databases that natively deal with collections of *JSON* documents).

Originally designed to deal with geo-tagging in *JSON* documents [15], *J-CO-QL*$^+$ (the query language of the *J-CO* Framework) has been enriched with powerful constructs for evaluating membership degrees of *JSON* documents to fuzzy sets and perform soft querying on them [9,10,12]. Indeed, we are revamping the former research on soft querying in databases, with the ultimate goal of overcoming the typical issue of those research works, i.e., papers have not been followed up by tools that could be practically applied in real-life scopes. To achieve this goal, we are defining constructs that allow users to define their own fuzzy concepts, within a uniform yet generic formal framework.

P. García Bringas et al. (Eds.): SOCO 2023, LNNS 749, pp. 56–68, 2023.
https://doi.org/10.1007/978-3-031-42529-5_6

Until now, an aspect of soft querying were not addressed in *J-CO-QL*$^+$, i.e., "aggregation of membership degrees". Suppose that *JSON* documents that already have associated membership degrees to fuzzy sets are grouped into one single *JSON* document (on the basis of field values): what is the membership degree of this new (group) document? Reasonably, it should be obtained by somehow aggregating membership degrees of formerly independent documents; however *J-CO-QL*$^+$ did not provide any support for defining user-defined "fuzzy aggregators", which are able to derive membership degrees from "arrays of membership degrees". This lack was particularly evident in [11], when we experimented how far *J-CO-QL*$^+$ was from providing an effective support for managing Intuitionistic Fuzzy Sets (an evolved form of fuzzy set [1]).

This paper provides two contributions. The first one is a meta-model for defining generic fuzzy aggregators. The second contribution is showing how to implement the meta-model within a stand-alone query language for *JSON* data sets, namely *J-CO-QL*$^+$, by adding novel constructs that remain coherent with its general philosophy. A plausible case study will show how to exploit, in practice, user-defined fuzzy aggregators to perform soft querying.

In the remainder, Section 2 provides the background. Section 3 formalizes the meta-model for fuzzy aggregators. Section 4 introduces the novel *J-CO-QL*$^+$ constructs in action, through a plausible case study. Finally, Sect. 5 draws the conclusions.

2 Background

In 1965, Zadeh [18] introduced "Fuzzy logic" to overcome the typical issues of classical Boolean logic, where an item either belongs or does not belong to a set, only. The idea of Zadeh was to exploit the concept of "fuzzy set", where the degree with which an x item belongs to an S set is expressed by a number (in the $[0, 1]$ range) that is said "membership degree" (or simply "membership", for short): 0 denotes that x does not belong at all to S; 1 denotes that x fully belongs to S; all intermediate values of membership denote a partial belonging of x to S. This way, it is possible to characterize qualitative evaluations given by people, as well as human sensations. Nonetheless, fuzzy sets are useful to perform vague or imprecise queries on data, on the basis of linguistic predicates, so as to perform soft querying of data [8,10,12].

What to do when we have to handle a large number of opinions, concerning the same subject, given by a group of people? This is a typical situation that occurs in decision making, when a group of experts have expressed their evaluations about alternative options. In this regard, "Fuzzy aggregators" are recognized as a powerful tool used in decision making; as reported in [16], opinions are expressed as fuzzy values; in order to make a decision, fuzzy aggregators allow for obtaining one single fuzzy value that can be used to rank alternatives. Some authors refer to this operation as "fusion" (see [4]).

Looking for fuzzy aggregators in the literature, a plethora of proposals can be found. However, many of them (such as "t-norm" and "t-conorm" operators,

see [6]) consider the aggregation of "pairs of items", for example either the same item belonging to two different fuzzy sets or two different items belonging to the same fuzzy set. The classical `AND` and `OR` operators, in the fuzzy version, are examples of this type of aggregators.

In this paper, we are focused on groups (or categories) G_j ($G_j = \{x_{j,1}, x_{j,2}, \dots\}$) of items $x_{j,i}$ that belong to the same group G_j because they share some common properties or are samples of the same category of items. Each $x_{j,i}$ singularly belongs to a fuzzy set A, thus it is provided with a membership $\mu_A(x_{j,i})$. Consequently, the set $\overline{A}$ of groups G_j can be seen as a partition of A: with this vision, the membership of a group G_j to $\overline{A}$ should be derived by somehow aggregating memberships $\mu_A(x_{j,i})$, group by group.

Focusing on this vision, we can select some very popular proposals.

The classical "Weighted aggregation" can be easily applied in multi-criteria decision making, for defining a scoring system to summarize many properties into a single one (see [5]). Given an array of memberships $\mathcal{M}$, a vector W of weights expresses the weight of each item in $\mathcal{M}$ ($\mathcal{M}$ and W have the same size); notice that no constraint is posed on the sum of weights; indeed, as shown in [3], it is always possible to normalize weights with respect to their sum.

The "Ordered Weighted Aggregation" (OWA) is a popular aggregation technique. It was proposed by [17] and later applied to aggregate memberships [13]. A monotone function $\phi(x) \in [0, 1]$ (for $x \in [0, 1]$) provides the weight of each single membership in the $\mathcal{M}$ vector, previously ordered in ascending or descending order. The weight w_i of the i-th item is $w_i = \phi(i/n) - \phi((i-1)/n)$.

Other fuzzy aggregators can be found in the literature [2,7]. For the sake of space we do not consider them here, but what is remarkable to point out, and it is the main contribution of this work, is that all these proposals lack a practical and effective stand-alone tool to be applied on real case studies.

3 Meta-Modeling Fuzzy Aggregators

In this section, we present the specific context of our work, so as to introduce the first contribution of this paper, i.e., a meta-model for fuzzy aggregators.

Context: the J-CO-QL$^+$ Data Model. In *J-CO-QL*$^+$, instructions work on "collections of *JSON* documents": given one or two input collections, instructions generate a new output collection. Each instruction can evaluate the membership of a document d to several fuzzy sets. The *J-CO-QL*$^+$ data model exploits a special root-level field named `~fuzzysets` to represent memberships. This field is used as a key/value map: the name of an internal field denotes a fuzzy-set name; its value is the membership to the fuzzy set.

In *J-CO-QL*$^+$, the `GROUP` statement groups together documents in the input collection based on the values of one or more "grouping fields": for each group, a single document gd_i is generated in the output collection, such that all grouped input documents $d_{i,j}$ are collected within an array field in gd_i. Clearly, if the $d_{i,j}$ documents have the `~fuzzysets` field, this is encompassed within the array.

How to compute the membership of gd_i to some fuzzy sets? By aggregating memberships of $d_{i,j}$ documents.

Meta-Model. We now provide the first contribution of this paper, i.e., a meta-model for fuzzy aggregators. The object of the aggregation is an "array of memberships" $\mathcal{M} = [\mu_1, \ldots, \mu_n]$, such that $|\mathcal{M}| = n$.

Definition 1. ***Derived Values.*** *For each item $\mathcal{M}[pos]$ (with $1 \leq pos \leq n$), a "pool of derived values" is a tuple*

$$\Delta(\mathcal{M}, pos, arg_1, arg_2, \ldots) = \langle\ \delta_1 : ldf_1(\mathcal{M}, pos, arg_1, arg_2, \ldots),\ \ldots,\ \delta_r : ldf_r(\mathcal{M}, pos, arg_1, arg_2, \ldots)\ \rangle$$

where δ_k (with $1 \leq k \leq r$) is a "derived-value name" (dvn).

$ldf_k(\mathcal{M}, pos, arg_1, arg_2, \ldots)$ is a function that derives the value for δ_k, i.e., $\delta_k = ldf_k(\mathcal{M}, pos, arg_1, arg_2, \ldots)$. Notice that both Δ and ldf_k receive both $\mathcal{M}$ (the array of memberships) and "pos" (the position in $\mathcal{M}$ on which to work); they also receive additional optional arguments of any type that can be necessary (for example, a weight vector).

Δ behaves as a key/value map: provided a name dvn,

$$\Delta(\mathcal{M}, pos, arg_1, arg_2, \ldots)[dvn] = ldf_k(\mathcal{M}, pos, arg_1, arg_2, \ldots),$$

where $k = indexOf(dvn)$ is the index of the $\delta_k = dvn$.

Through Δ, it is possible to derive values that will be useful for defining complex aggregations.

Definition 2. ***Aggregated Values.*** *Given $\mathcal{M}$ (the array of memberships) and Δ (the pool of derived values), the "pool of aggregated values" is defined as*

$$\Theta(\mathcal{M}, arg_1, arg_2, \ldots) = \langle \theta_1 : laf_1(\mathcal{M}, pos, arg_1, arg_2, \ldots), \ldots, \theta_t : laf_t(\mathcal{M}, pos, arg_1, arg_2, \ldots) \rangle$$

where θ_l (with $1 \leq l \leq t$ and $t > 0$) is an "aggregated-value name" (avn).

The value of a θ_l aggregated value is obtained by aggregating the result of the associated "local aggregation function" $laf_l(\mathcal{M}, pos, arg_1, arg_2, \ldots)$, i.e.,

$$\theta_l = \Sigma_{pos=1}^{n} laf_l(\mathcal{M}, pos, arg_1, arg_2, \ldots).$$

In other words, for each item in the $\mathcal{M}$ array of memberships, the laf_l function is evaluated; the results are aggregated by summing them. Notice that a laf function can exploit the pool Δ of derived values.

Definitely, $\Theta(\mathcal{M}, arg_1, arg_2, \ldots)$ behaves as a key/value map: given a name avn, $\Theta(\mathcal{M}, arg_1, arg_2, \ldots)[avn] = \theta_l$, where $l = indexOf(avn)$ is the index of the $\theta_l = avn$. Consequently, we can write

$$\Theta(\mathcal{M}, arg_1, arg_2, \ldots)[avn] = \Sigma_{pos=1}^{n} laf_l(\mathcal{M}, pos, arg_1, arg_2, \ldots).$$

The rationale behind Θ is that several aggregated values can be obtained for one single array of memberships: indeed, fuzzy aggregators could provide the final membership by combining several aggregated values.

Definition 3. ***Fuzzy Aggregator.*** *Given $\mathcal{M}$ (array of memberships), Δ (pool of derived values) and Θ (pool of aggregated values), a "fuzzy aggregator" fAggr is a function $fAggr(\mathcal{M}, arg_1, arg_2, \ldots) = limit(e(\mathcal{M}, arg_1, arg_2, \ldots)) \in [0, 1]$.*

$e(\mathcal{M}, arg_1, arg_2, \dots)$ is an expression that refers to aggregated-value names in $\Theta(\mathcal{M}, arg_1, arg_2, \dots)$. The resulting r value is a membership, but the e expression could return a value outside the $[0,1]$ range: the limit function ensures that the resulting value is in the $[0,1]$ range. Specifically, if $r < 0$, then $limit(r) = 0$; if $r > 1$, then $limit(r) = 1$; if $0 \leq r \leq 1$, then $limit(r) = r$.

Consider a collection C of *JSON* documents and a document $d_i \in C$. If we want to obtain the membership of d_i to a fuzzy set F through the *fAggr* fuzzy aggregator, denoted as $\mu_F(d_i)$, it is

$\mu_F(d_i) = fAggr(\mathcal{M}(d_i), arg_1, arg_2, \dots)$,

where $\mathcal{M}(d_i)$ is an array of membership extracted either from an array of nested documents in d_i or from the `~fuzzysets` field in d_i.

Hereafter, we provide a couple of examples of commonly-used aggregators.

Example 1. Consider an "Ordered Weighted Aggregation" (OWA, for short) on an array $\mathcal{M}$ of memberships, which we assume sorted in ascending order. We have to consider a "weighting function" $\phi(x)$, which is used to obtain the weight of each single item in $\mathcal{M}$; for example, we can choose $\phi(x) = x^2$, so that the weight for each single item in $\mathcal{M}$ is $w[pos] = (pos/|\mathcal{M}|)^2 - ((pos-1)/|\mathcal{M}|)^2$, so as to give highest weight to highest values. The aggregator is formally defined as:

$\Delta(\mathcal{M}, pos) = \langle w : (pos^2 - (pos-1)^2/|\mathcal{M}|^2 \rangle$,
$\Theta(\mathcal{M}) = \langle av : \Delta(\mathcal{M}, pos)[w] \times \mathcal{M}[pos] \rangle$,
$OWA(\mathcal{M}) = \Theta(\mathcal{M})[av]$.

For each item in $\mathcal{M}$, one derived value, named w, is computed (see Δ): it is the weight of the item in position *pos*. One single aggregated value, named av (see Θ), is computed, by summing, for each item, the product of its value by its weight w (from Δ). The av aggregated value is the membership computed by the OWA fuzzy aggregator.

Example 2. As a second example, we consider the case in which the $\mathcal{M}$ array is accompanied by an array of weights, denoted as W. In this case, the W argument is added to functions that define the fuzzy aggregator, named *Weigher*.

$\Delta(\mathcal{M}, pos, W) = \langle wi : W[pos] \times \mathcal{M}[pos] \rangle$
$\Theta(\mathcal{M}, W) = \langle av : \Delta(\mathcal{M}, pos, W)[wi], aw : W[pos] \rangle$
$Weigher(\mathcal{M}, W) = \Theta(\mathcal{M}, W)[av] / \Theta(\mathcal{M}, W)[aw]$.

For each item in $\mathcal{M}$, its value is multiplied by the corresponding weight in the W array; the resulting value is named wi (which stands for "weighted item", see Δ). Two aggregated values are defined. The first one is named av; it is computed by summing the wi derived values computed by Δ. The second aggregated value is named aw (which stands for "aggregated weights", see Θ); it is computed by summing the values in the W array.

The *Weigher* fuzzy aggregator obtains the aggregated membership by dividing the av aggregated value by the aw aggregated value. This example shows the usefulness of defining several aggregated values.

(a) Example of document in the source `ProductEvalations` collection.

(b) Example of fuzzified document generated by Line 6.

Fig. 1. *JSON* documents processed by the *J-CO-QL*$^+$ script. (Color figure online)

4 Novel Constructs in *J-CO-QL*$^+$ and Case Study

This section provides the second contribution of this paper, i.e., showing how the meta-model for fuzzy aggregators can be implemented in a query language and how can be used in practice. To this end, we exploit a plausible case study.

4.1 Case Study

Suppose that a market-research company, named *ACME*, leads a survey to measure the popularity of a set of products among consumers. Customers give their opinion on a set of properties of a product by providing an integer score in the $[1, 10]$ range (1 indicates *no appreciation at all*, while 10 indicates the *most appreciation*). The properties to evaluate are (i) the *appeal of the brand*, (ii) the *convenience of the price* and (iii) the *general quality* of the product.

For each product and for each interview, *ACME* stores the data in a *JSON* document. Figure 1a shows an example: the `id` field reports the identifier of the interview, while the `productId` and `productName` fields report, respectively, the identifier and the name of the product. Then, the fields named (i) `brand`, (ii) `price` and (iii) `quality` report the evaluations provided by the interviewed customer related to his/her opinion about, respectively, (i) the appeal of the brand, (ii) the convenience of the price and (iii) the quality of the product.

At the end of the survey, *ACME* obtains a collection of documents similar to the one shown in Fig. 1a, on which to perform the query reported hereafter.

Query 1. *Discover those products with the highest rank on the basis of opinions provided by customers.*

This is a query that can be performed with a soft approach that relies on fuzzy aggregators. In the reminder of the section, we present the *J-CO-QL*$^+$ script that provides an answer to Query 1, by presenting its constructs in action. Due to its length and complexity, the *J-CO-QL*$^+$ script is divided in two parts:

Listing 1 (discussed in Sect. 4.2) defines the fuzzy concepts to apply later; Listing 2 (discussed in Sect. 4.3) actually performs the soft query.

Listing 1 *J-CO-QL*$^+$ script: Part 1.

```
1. CREATE FUZZY OPERATOR fuzzifyEvaluation
     PARAMETERS   eval TYPE INTEGER
     PRECONDITION eval IN_RANGE [1,10]
     EVALUATE     (eval-1) / 9;

2. CREATE FUZZY AGGREGATOR owa
     PARAMETERS   M TYPE ARRAY ASC
     FOR ALL      x IN M
       LOCALLY    (POS^2 -(POS-1)^2) / (COUNT (M)^2) AS w
       AGGREGATE  x * w AS av
     EVALUATE     av;

3. CREATE FUZZY AGGREGATOR weigher
     PARAMETERS
       M          TYPE ARRAY,
       weights    TYPE ARRAY
     PRECONDITION COUNT (M) = COUNT (weights)
     FOR ALL      x IN M
       LOCALLY    x * weights[POS] AS wi
       AGGREGATE  wi AS av
       AGGREGATE  weights[POS] AS aw
     EVALUATE     av / aw;
```

4.2 Declaring Fuzzy Operator and Fuzzy Aggregators

The goal of Part 1 of the *J-CO-QL*$^+$ script, reported in Listing 1, is to declare the Fuzzy Operator and the Fuzzy Aggregators that are used in Listing 2 for soft querying data. Hereafter, we present Listing 1 in details.

Line** 1**: creating a fuzzy operator. The `CREATE FUZZY OPERATOR` instruction defines a fuzzy operator named `fuzzifyEvaluation`. In *J-CO-QL*$^+$ (see [12]), a "fuzzy operator" actually computes memberships from document fields, so as to obtain the membership of documents to fuzzy sets.

Specifically, the goal of the `fuzzifyEvaluation` operator is to turn a single property evaluation (provided by a customer) into a membership to a fuzzy set. We present the operator in details.

- The `PARAMETER` clause defines the integer `eval` parameter to fuzzify.
- The optional `PRECONDITION` clause checks that the actual value of the `eval` parameter is in the $[1, 10]$ range.
- Finally, the `EVALUATE` clause provides the mathematical expression to turn the `eval` parameter into a membership degree returned by the fuzzy operator. Notice that, when the value of the `eval` parameter is equal to 1, the membership is 0 (e.g., full non-membership); when the value of the `eval` parameter is equal to 10, the membership is 1 (e.g., full membership).

Line** 2**: creating a fuzzy aggregator. The `CREATE FUZZY AGGREGATOR` instruction is the novel instruction introduced in the *J-CO-QL*$^+$ language to define fuzzy aggregators: it implements the meta-model introduced in Sect. 3. In Line 2 of Listing 1, a fuzzy aggregator named `owa` is defined. Its goal is to aggregate an array of memberships, so as to return a weighted sum of them, in such a way that the greatest memberships have the greatest weights (see Example 1).

- The `PARAMETER` clause defines the input `M` array of memberships to aggregate. The `ASC` option specifies that the `M` array is sorted in ascending order before being evaluated.
- The `FOR ALL` clause allows for deriving values in the sense of Definition 1 in Sect. 3. In this case, for each `x` element in the `M` array, the `LOCALLY` expression computes a weight, named `w`, whose value is given on the basis of the position of the `x` element in the `M` array, by means of the `POS` pre-defined symbol (the first item in an array has `POS` = 1), and the number of elements in the `M` array, provided by the `COUNT` built-in function. Notice that, according to the given definition, the last computed `w` weights are greater than the first computed `w` weights, and that the sum of all `w` weights is 1.
  As an example, if we have an array `M` of 5 memberships, the sequence of generated weights is (the first number in the pairs is the position `POS`, while the second number is the weight)
  $$(1, 0.04),\ (2, 0.12),\ (3, 0.20),\ (4, 0.28),\ (5, 0.36).$$
- The `AGGREGATE` clause specifies how to compute the `av` aggregated value; specifically, it sums the product of the `x` item by the `w` derived value computed for all the `x` items in `M`. Indeed, `av` is the weighted sum of all `x` items.
- Finally, the `EVALUATE` clause returns the `av` aggregated value as final result of the fuzzy aggregator. Notice that, since the `M` array is an array of memberships, the `av` value is in the $[0, 1]$ range, by construction.

Line** 3**: creating a second fuzzy aggregator. A further fuzzy aggregator, named `weigher`, is defined. Its goal is to weigh an array of memberships on the basis of an array of weights (see Example 2). Hereafter, we present it in details.

- The `PARAMETER` clause defines two input arrays: the `M` array contains the memberships to aggregate; the `weights` array contains the weights to use to aggregate the `M` array.
- The optional `PRECONDITION` clause checks that the `M` array and the `weights` arrays have the same number of elements. If the condition is not satisfied, the aggregator cannot proceed with its task.
- The `FOR ALL` clause derives, for each `x` item in the `M` array, its weighted value, named as `wi`, by means of the `LOCALLY` expression. Notice the use of the `POS` symbol to access the element in the `weights` array that corresponds to the `x` item (indeed, `x` is a shortcut for `M[POS]`). Then, the first `AGGREGATE` clause sums all the weighted values `wi` into the `av` value. The second `AGGREGATE` clause sums all the elements in the `weights` array into the `aw` value.
- Finally, the `EVALUATE` clause returns the `av` value normalized by the `aw` value, as final result of the fuzzy aggregator.

Referring to the meta-model presented in Sect. 3, `LOCALLY` clauses correspond to the pool Δ of derived values, while `AGGREGATE` clauses correspond to the pool Θ of aggregated values.

4.3 Soft Querying

Part 2 of the *J-CO-QL*$^+$ script (reported in Listing 2) performs the soft query in three steps: (i) the opinions in *ACME*'s collection are fuzzified; (ii) for each product, all the fuzzified opinions are grouped together; (iii) for each product, a ranking value is computed. Hereafter, we present Listing 2 in details.

Line** 4**: connecting to a database. The `USE DB` instruction connects to a database named `Soco2023` that is managed by a *J-CO-DS* server (*J-CO-DS* is the *NoSQL* database provided within the *J-CO* Framework [14]).

Line** 5**: loading data. The `GET COLLECTION` instruction retrieves a collection of *JSON* documents, named `ProductEvaluations`, containing the answers to *ACME*'s survey. Figure 1a shows an example of document in the collection.

Line** 6**: fuzzifying data. The role of the `FILTER` instruction on this line is to fuzzify the evaluations in each document of the current collection.

- The `CASE WHERE` condition selects only those documents holding, at root-level, the `productId` and `productName` fields.
- The first `FUZZY SET` branch in the `CHECK FOR` clause (within the `GENERATE` section) evaluates the membership of the document to the `Appeal` fuzzy set. The membership is computed by the `USING` soft condition that exploits the `fuzzifyEvaluation` fuzzy operator defined on Line 1 to which the `brand` field is passed as actual parameter.
- Similarly, the second `FUZZY SET` branch evaluates the membership of the document to the `Cheapness` fuzzy set. This time, the membership is computed passing the `price` field to the `fuzzifyEvaluation` fuzzy operator.
- Finally, the third `FUZZY SET` branch evaluates the membership to the fuzzy set named `Perception`, by passing the `quality` field as actual parameter.

Figure 1b reports how the `FILTER` instruction transforms the document shown in Fig. 1a. Notice the novel `~fuzzysets` field (highlighted by a blue box).

Line** 7**: grouping data. The `GROUP` instruction creates a new collection by grouping the documents in the current collection.

- Specifically, the `PARTITION` clause selects only those documents having the `productId` and `productName` fields.
- The following `BY` clause groups all documents with the same value for the `productId` and `productName` fields. For each group, a new document is generated with the grouping fields at root-level and a new array field named `evaluations`, as specified by the `INTO` clause, that holds all the documents in the group.
- The `DROP GROUPING FIELDS` option specifies that grouping fields must be removed from the documents inside the `evaluations` array.

Figure 2a reports an example of grouped document. Notice that the array field named `evaluations` holds the document shown in Fig. 1b (highlighted by

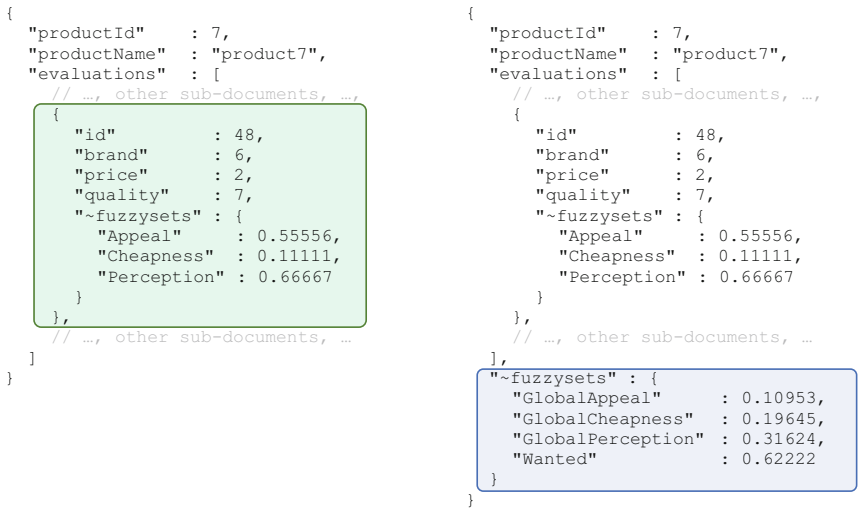
a green box), from which the `productId` and `productName` grouping fields were removed.

Listing 2 *J-CO-QL*$^+$ script: Part 2.

```
4. USE DB Soco2023
     ON SERVER jcods 'http://127.0.0.1:17017';

5. GET COLLECTION ProductEvaluations@Soco2023;

6. FILTER
     CASE WHERE WITH .productId, .productName
       GENERATE
         CHECK FOR
           FUZZY SET Appeal
             USING fuzzifyEvaluation (.brand),
           FUZZY SET Cheapness
             USING fuzzifyEvaluation (.price),
           FUZZY SET Perception
             USING fuzzifyEvaluation (.quality);

7. GROUP
     PARTITION WITH .productId, .productName
       BY     .productId, .productName
       INTO   .evaluations
         DROP GROUPING FIELDS;

8. FILTER
     CASE WHERE WITH .productId, .productName
       GENERATE
         CHECK FOR
           FUZZY SET GlobalAppeal
             USING AGGREGATE THROUGH
               owa (MEMBERSHIP_TO Appeal FROM ARRAY .evaluations),
           FUZZY SET GlobalCheapness
             USING AGGREGATE THROUGH
               owa (MEMBERSHIP_TO Cheapness FROM ARRAY .evaluations),
           FUZZY SET GlobalPerception
             USING AGGREGATE THROUGH
               owa (MEMBERSHIP_TO Perception FROM ARRAY .evaluations),
           FUZZY SET Wanted
             USING AGGREGATE THROUGH
               weigher(MEMBERSHIP_TO [ GlobalAppeal, GlobalCheapness,
                                       GlobalPerception ], [2, 3, 5])
         ALPHACUT 0.5 ON Wanted
         BUILD {
           .productId   : .productId,
           .productName : .productName,
           .rank        : MEMBERSHIP_TO (Wanted)
         }
         DEFUZZIFY;

9. SAVE AS RankedProducts@Soco2023;
```

Line** 8**: soft querying data. The second `FILTER` instruction ranks the grouped documents that now constitute the current collection.

- The `CASE WHERE` clause selects only those documents with the `productId` and `productName` fields.
- The first `FUZZY SET` branch in the `CHECK FOR` clause evaluates the membership of a document to the `GlobalAppeal` fuzzy set, through the `AGGREGATE THROUGH` construct. The membership is computed by exploiting the `owa` fuzzy aggregator defined on Line 2. The `MEMBERSHIP_TO` operator extracts the memberships to the `Appeal` fuzzy set of documents within the `evaluations` array (its syntax is `MEMBERSHIP_TO` *fuzzySetName* `FROM` *fieldRef*). The `owa` aggregator receives this array of memberships and aggregates them; the resulting aggregated membership degree becomes the membership to the novel `GlobalAppeal` fuzzy set.



(a) Example of grouped document generated by Line 7.

(b) Example of document generated by Line 8, before `BUILD` block.

Fig. 2. Documents obtained after grouping the fuzzified source documents.

- Similarly, the second `FUZZY SET` branch evaluates the membership of the document to the `GlobalCheapness` fuzzy set, by aggregating memberships of documents within the `evaluations` array to the `Cheapness` fuzzy set, trough the `owa` aggregator.
- The third `FUZZY SET` branch evaluates the membership of the document to the `GlobalPerception` fuzzy set, by aggregating the memberships to the `Perception` fuzzy set of documents within the `evaluations` array field, trough the `owa` aggregator.
- The last `FUZZY SET` branch evaluates the membership of a document to the `Wanted` fuzzy set by exploiting the `weigher` fuzzy aggregator defined on Line 3. This time, the `MEMBERSHIP_TO` operator reports only an "array of fuzzy-set names", meaning that the array of memberships is built by acquiring the memberships to the specified fuzzy sets of the current document. The array of memberships so far built is passed to the `weigher` operator, together with the array of weights.
- The `ALPHACUT` option keeps all documents with the membership to the `Wanted` fuzzy set no less than 0.5.
- The `BUILD` block restructures the document by keeping the `productId` and the `productName` fields, also by adding the novel `rank` field whose value is given by the membership to the `Wanted` fuzzy set. Automatically, the `BUILD` block removes the unreported fields (e.g., the `evaluations` field).
- Finally, the `DEFUZZIFY` option defuzzifies the document by removing the special `~fuzzysets` field. Figure 3 shows an example of final document, obtained by processing the example documents in Fig. 2a and Fig. 2b.

```
{
  "productId"    : 7,
  "productName"  : "product7",
  "rank"         : 0.62222
}
```

Fig. 3. Example of final document generated by Line 8.

Line** 9**: saving data. Finally, the `SAVE AS` instruction saves the resulting collection into a novel collection named `RankedProducts` in the `Soco2023` database.

Notice that Line 8 exploits other two novel constructs that have been introduced in *J-CO-QL*$^+$ to deal with aggregators. These constructs are `AGGREGATE THROUGH` and `MEMBERSHIP_TO`.

5 Conclusions

In this paper, we addressed the topic of defining a meta-model for fuzzy aggregators and implementing it within the *J-CO* Framework.

This is a tile in the research activity we are carrying on of revamping former research on soft querying in databases, but working on modern *JSON* document stores: to develop the research in an effective way, a practical tool is essential to experiment and validate ideas; thus, we are implementing our ideas within the *J-CO* Framework. *J-CO-QL*$^+$, the query language of the *J-CO* Framework, is undergoing a continuous evolution, so as to provide potential users with a robust tool for performing soft querying on *JSON* data sets.

This paper addresses the missing tile of user-defined fuzzy aggregators. The first contribution is the definition of a meta-model for defining a wide range of fuzzy aggregators (presented in Sect. 2). The second contribution is showing that it is possible to implement the meta-model in a stand-alone query language for collections of *JSON* documents, by exploiting user-defined fuzzy aggregators in a plausible case study.

As a future work, we plan to introduce several extensions to the *J-CO* Framework, so as to broaden its potential range of application. Currently, we are addressing *multi-grade fuzzy-set models*, through the definition of a generalized model to let users of the *J-CO* Framework use their own-defined fuzzy-set models. We also plan to prepare a library of *J-CO-QL*$^+$ scripts with the definitions for the most common fuzzy aggregators, allowing users to incorporate and use them within their soft queries.

Definitely, our aim is to make the *J-CO* Framework available for the community on a public GitHub repository[1].

[1] Github repository of the *J-CO* Framework: https://github.com/JcoProjectTeam/JcoProjectPage.

References

1. Atanassov, K.: Intuitionistic fuzzy sets. Fuzzy Sets Syst. **20**, 187–96 (1986)
2. Bordogna, G., Psaila, G.: Soft aggregation in flexible databases querying based on the vector p-norm. I. J. Uncertainty Fuzziness Knowl. Based Syst. **17**(Suppl. 01), 25–40 (2009)
3. van den Broek, P., Noppen, J.: Fuzzy weighted average: alternative approach. In: 2006 Annual Meeting of the North American Fuzzy Information Processing Society, NAFIPS 2006, pp. 126–130. IEEE (2006)
4. Calvo, T., Mesiar, R., Yager, R.R.: Quantitative weights and aggregation. IEEE Trans. Fuzzy Syst. **12**(1), 62–69 (2004)
5. Dombi, J., Jónás, T.: Weighted aggregation systems and an expectation level-based weighting and scoring procedure. Eur. J. Oper. Res. **299**(2), 580–588 (2022)
6. Farahbod, F., Eftekhari, M.: Comparison of different t-norm operators in classification problems. arXiv preprint arXiv:1208.1955 (2012)
7. Fodor, J.: Aggregation functions in fuzzy systems. In: Fodor, J., Kacprzyk, J. (eds.) Aspects of Soft Computing, Intelligent Robotics and Control. Studies in Computational Intelligence, vol. 241. Springer, Heidelberg (2009). https://doi.org/10.1007/978-3-642-03633-0_2
8. Fosci, P., Marrara, S., Psaila, G.: GeoSoft: a language for soft querying features within GeoJSON information layers. In: Marchiori, M., Domínguez Mayo, F.J., Filipe, J. (eds.) Web Information Systems and Technologies, WEBIST WEBIST 2020 2021. LNBIP, vol. 469, pp. 196–219. Springer, Cham (2023). https://doi.org/10.1007/978-3-031-24197-0_11
9. Fosci, P., Psaila, G.: *J-CO*, a framework for fuzzy querying collections of *JSON* documents (demo). In: Andreasen, T., De Tré, G., Kacprzyk, J., Legind Larsen, H., Bordogna, G., Zadrożny, S. (eds.) FQAS 2021. LNCS (LNAI), vol. 12871, pp. 142–153. Springer, Cham (2021). https://doi.org/10.1007/978-3-030-86967-0_11
10. Fosci, P., Psaila, G.: Towards flexible retrieval, integration and analysis of JSON data sets through fuzzy sets: a case study. Information **12**(7), 258 (2021)
11. Fosci, P., Psaila, G.: Intuitionistic fuzzy sets in J-CO-QL$^+$? In: García Bringas, P., et al. (eds.) 17th International Conference on Soft Computing Models in Industrial and Environmental Applications, SOCO 2022. LNNS, pp. 134–145 vol. 531. Springer, Cham (2023). https://doi.org/10.1007/978-3-031-18050-7_13
12. Fosci, P., Psaila, G.: Soft integration of geo-tagged data sets in J-CO-QL$^+$. ISPRS Int. J. Geo Inf. **11**(9), 484 (2022)
13. Li, H., Yen, V.C.: Fuzzy Sets and Fuzzy Decision-Making. CRC Press (1995)
14. Psaila, G., Fosci, P.: Toward an anayist-oriented polystore framework for processing JSON geo-data. In: International Conference on Applied Computing 2018, Budapest, Hungary, 21–23 October 2018, pp. 213–222. IADIS (2018)
15. Psaila, G., Fosci, P.: J-CO: a platform-independent framework for managing georeferenced JSON data sets. Electronics **10**(5), 621 (2021)
16. Vaníček, J., Vrana, I., Aly, S.: Fuzzy aggregation and averaging for group decision making: a generalization and survey. Knowl. Based Syst. **22**(1), 79–84 (2009)
17. Yager, R.R.: On ordered weighted averaging aggregation operators in multicriteria decision making. IEEE Trans. Syst. Man Cybern. **18**(1), 183–190 (1988)
18. Zadeh, L.A.: Fuzzy sets. Inf. Control **8**(3), 338–353 (1965)